

作业1

24371277 刘哲晗

问题1

关于要求3：写者优先于读者（一旦有写者，则**后续读者必须等待**，唤醒时优先考虑写者）。理解：阻挡后续读者，直到前面读者全部读完释放rmutex后，写者拿取 写操作结束后 释放enter给后面的读写者。

- 与读写互斥不是一回事，后者是处理对资源的竞争，前者是“进程调度” 写者会阻挡后续读者。

```
int count = 0;
semaphore rmutex = 1; //读（count）变量操作互斥
semaphore rwmutex = 1; //读写互斥
semaphore enter = 1; //调度读写者，有写者进入，

int main()
{
    while(1){
        writer();
        reader();
    }
}

writer(){
    P(enter);
    P(rwmutex);
    写文件
    V(rwmutex);
    V(enter);
}

reader(){
    P(enter); //读写公平，允许写插队
    P(rmutex);
    if(count == 0) P(rwmutex);
    count++;
    V(rmutex);
    V(enter);

    读文件;

    P(rmutex);
    count--;
    if(count == 0) V(rwmutex);
    V(rmutex);
}
```

问题2

寿司店问题。假设一个寿司店有5个座位，如果你到达的时候有一个空座位，你可以立刻就坐。但是如果你到达的时候5个座位都是满的有人已经就坐，这就意味着这些人都是一起来吃饭的，那么你需要等待所有的人一起离开才能就坐。编写同步原语，实现这个场景的约束。

注意1：就坐条件：5个人已满或者**存在等待中的人**，因此实质上有一个等待队列。

- 当最后一个顾客走了后，最多唤醒下一批5个人进来

注意2：判断条件，绝对不能判断信号量的值，因为信号量的操作只有PV，**需要设置普通变量**

提高3：判断"一批结束"，永远用"正在做的人的计数归0"来判断，不要用"剩下的空位满了"来判断。

提高4：普通变量的访问，判断，修改，都需要互斥锁上锁（最小的锁）

```
#define MAXN 5
int seats = MAXN;
int waiting = 0; //等待顾客数!
int eating = 0; //正在吃饭的顾客数
semaphore mutex = 1; //座位判断 需要互斥保护
semaphore can_enter = 0; //同步 门票
Customer()
{
    while(1)
    {
        P(mutex);
        if(seats == 0 || waiting > 0)
        {
            waiting++;
            V(mutex);
            P(can_enter);
            //坐座位
            P(mutex);
            waiting--;
        }

        seats--;
        eating++;
        V(mutex);

        eat();

        P(mutex);
        seats++;
        eating--;

        //全部离开了，无论之前是否是5个人一起，都要唤醒所有等待中的顾客（小于座位数
        if(eating == 0){
            for(int i = 0; i < waiting && i < N; i++)
            {
                V(can_enter);
            }
        }
        V(mutex);
    }
}
```

```
}  
}
```

问题3

(1) P、V操作和信号量的物理意义

信号量：受保护的共享资源计数器 + 进程等待队列，实现进程同步与互斥

- 互斥型（如 `mutex`）：代表临界资源可用数量，初始通常为1
- 同步型（如 `can_enter`）：代表前驱条件是否满足，初始通常为0
- $S > 0$ ：有 S 个可用资源； $S = 0$ ：资源被占； $S < 0$ ：有 $|S|$ 个进程在等待

P操作（申请）：原子执行 $S = S - 1$

- $S \geq 0$ ：申请成功，进程继续
- $S < 0$ ：无可用资源，进程阻塞进入等待队列

V操作（释放）：原子执行 $S = S + 1$

- $S > 0$ ：无等待进程，继续执行
- $S \leq 0$ ：唤醒等待队列中的第一个进程

(2)

2) 一个软件公司有5名员工，每人刷卡上班。员工刷卡后需要等待，直到所有员工都刷卡后才能进入公司。为了避免拥挤，公司要求员工一个一个通过大门。所有员工都进入后，最后进入的员工负责关门。请用P、V操作实现员工之间的同步关系。

- 屏障模板：

```
int card_remained = 5; //剩余待刷卡人数  
int people_remained = 5; //剩余待进入人数  
semaphore gates = 1; //进入大门互斥  
semaphore perm = 0; //所有员工都刷卡了  
semaphore mutex = 1;  
  
Employee()  
{  
    //--barrier模型  
    P(mutex);  
    card_remained--; //刷卡  
  
    if(card_remained == 0) //都刷完了  
    {  
        for(int i = 0; i < 5; i++) //通知所有人  
        {  
            V(perm);  
        }  
    }  
}
```

```

V(mutex);
P(perm); //!让所有人拿到权限

//同步
P(gates);
P(mutex);
people_reained--; //进门
if(people_reained == 0)
{
    V(mutex);
    close_the_gate();
    V(gates);
}
else
{
    V(mutex);
    V(gates);
}
}

```

问题4

注意1：多个搜索进程可以并发，必须参考读者写者模式 这样做会导致无法并发

```

find()
{
    P(fi_de); //d
    ... //读取链表，查找
    V(fi_de);
}

```

- 需要添加**读者计数器**，第一位读者来的时候上与其他进程的互斥锁，

```

semaphore in_in = 1;
semaphore de_de = 1;
semaphore in_de = 1;
semaphore fi_de = 1;
int count = 0; //读者数
semaphore fi_mutex = 1; //保护搜索计数的互斥锁
int main()
{
    while(1)
    {
        find();
        insert();
        delete();
    }
}

```

```

}
find()
{
    P(fi_mutex);
    count++;
    if(count > 0) P(fi_de);
    V(fi_mutex);
    ... //读取链表, 查找
    P(fi_mutex);
    count--;
    if(count == 0) V(fi_de);
    V(fi_mutex);
}
insert()
{
    P(in_de);
    P(in_in);
    ... //插入链表操作
    V(in_in);
    V(in_de);
}
Delete(){
    P(in_de);
    P(fi_de);
    P(de_de);
    ... //删除链表某项的操作
    V(in_de);
    V(fi_de);
    V(de_de);
}

```

提升方向 类似问题1，查找优先于删除，如果读者源源不断，会导致删除饿死。（加一个新的进入信号即可）

问题5

5. 公园购票问题。在五一期间，某公园实行限流访问，规定每天最多允许 N 个人购票入园。公园的在线售票系统提供了查询余票和在线购票功能，查询者 Q 和购票者 B 分别对余票数量进行读操作和写操作。为了简化问题，假定每次只能买一张票。请基于 PV 操作设计一个算法，实现多个 Q 和 B 对余票数量的并发访问。要求:(1)Q 和 B **按照到达顺序** 对余票数量进行访问;(2)当多个 Q 连续到达时，应允许它们**并发读取**余票数量;(3) 当余票为 0 时，不允许 B 执行写操作。

- 核心原则：对共享变量的判断和修改，必须在同一个临界区内完成！（访问共享变量需要上锁）

```

int q_count = 0;
int remained = N;
semaphore enter = 1;
semaphore w_mutex = 1;
semaphore r_mutex = 1;
main()
{
    while (1){

```

```
        query();
        buy();
    }
}

buy()
{
    P(enter);
    P(w_mutex);

    if(remained == 0){
        printf("没有票了");
        V(w_mutex);
        V(enter);
        return;
    }

    remained--;
    ...//买票
    V(w_mutex);
    V(enter);
}

query()
{
    P(enter);
    P(r_mutex);
    if(q_count == 0) P(w_mutex);
    q_count++;
    V(r_mutex);
    V(enter);

    ...//查询余票

    P(r_mutex)
    q_count--;
    if(q_count == 0) V(w_mutex);
    V(r_mutex);
}
```